

Reproducibility Report: Graph Edit Networks Mostly Hold Up — But One Scaling Claim Doesn't

A University of Ljubljana reproduction re-implements the model, checks all four experimental claims, and asks what the synthetic benchmarks actually test

A quiet but practical problem in graph machine learning is predicting how a graph changes over time. Given a sequence of graphs that evolves step by step, you want to forecast the next one — and not just as a label, but as the specific set of structural changes that turn the current graph into the next: insert a node, delete an edge, relabel a vertex.

Standard graph neural networks are an awkward fit for this. They emit probability scores over nodes or edges, not the discrete structural operations that actually transform one graph into another. Graph Edit Networks (GEN) were proposed to close that gap: an output layer that sits on top of a GNN and predicts an explicit, human-readable edit script instead of a score.

This post is not about the original GEN paper but about an independent reproduction of it. The authors re-implement GEN from scratch, check each of the paper's four experimental claims one by one, and — crucially — write down the synthetic data generators that the original paper described only in passing. The verdict is mixed: three of the four claims survive, but the claim that backward passes scale linearly does not. More pointedly, the reproduction finds that several of the benchmarks reward a model for revisiting transformations it has already seen, rather than for genuinely generalizing.

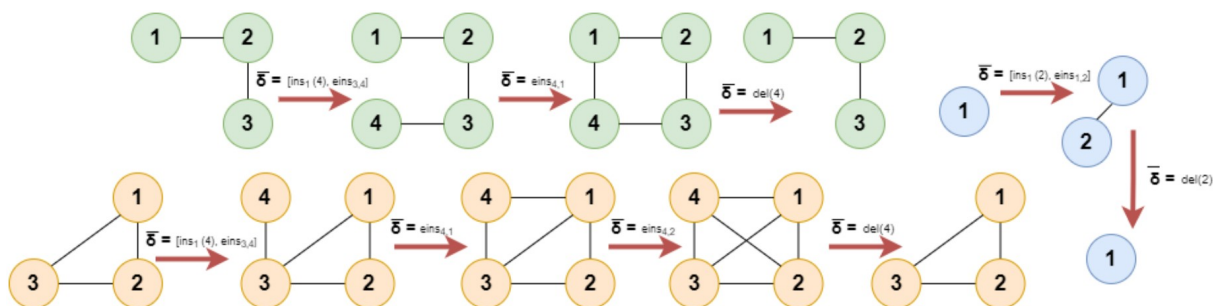


Figure 1. Three cyclical time series produced by the Edit Cycles data generator. Each transition is written as an edit operation δ — ins to insert a node, eins to insert an edge, del to delete a node — and each red arrow is a single graph edit. This is exactly what GEN learns to predict: not a static graph, but the script of operations that maps the current graph to the next.

Paper: <https://doi.org/10.5281/zenodo.6574701> (ReScience C 2022, ML Reproducibility Challenge 2021)

Code: https://github.com/MarusaOrazem/reproducibility_challenge

The problem: predicting how a graph changes, not what it is

Many real systems are graphs that move: molecules break and form bonds during a reaction, ties in a social network come and go, a logical expression gets rewritten as it is simplified. To predict their next state, a model needs to output a set of structural edits, not a heatmap of node or edge probabilities. Forcing 'the next graph' into a node-or-edge scoring framework tends to be both unintuitive and hard for a human to interpret afterwards.

GEN changes the prediction target to the edit script itself. It assumes graph evolution is Markovian: the next graph depends only on the current one. The model therefore only has to predict a finite string of edits for the current graph, which — applied in order — yields the next graph.

The method: turning a prediction into a sequence of graph edits

GEN attaches to a standard GNN backbone, and instead of scores it emits an edit script drawn from a small vocabulary of operations: insert, delete, or replace a node, and insert or delete an edge. Applying that script in sequence to the current graph produces the next one — precisely the Markovian assumption the method rests on.

The training signal comes from reference mappings between time-adjacent graphs. Those mappings are obtained with graph-edit-distance (GED) approximators, because computing the exact distance is NP-hard, and are then converted into a per-node teaching signal. The reproduction trains two loss variants — an adapted hinge loss and an adapted cross-entropy loss — and, for large citation graphs, adds edge-filtering schemes that cap how many edge edits each node may propose. The baseline is a modified Variational Graph Autoencoder (VGAE).

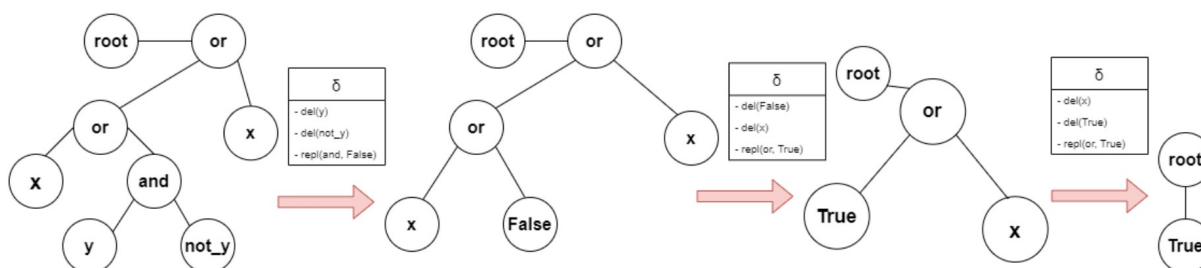


Figure 2. One tree evolution from the Boolean Formulae generator: the leftmost graph is the syntax tree of the logical formula $(x \vee (y \wedge \neg y)) \vee x$, and each subsequent step is a logical simplification of the previous graph. Each δ box lists the edits for that step (del to delete, repl to replace). GEN's job is to predict such a script given the current tree.

What the reproduction did: rewrite the code, document the data, change the protocol

The value here goes well beyond re-running the numbers. The authors re-implement GEN in Python, check that the code matches the method as described (fixing a section of the original authors' code along the way), and thoroughly document the five synthetic data-generating processes (DGPs) that the original paper only sketched. The five fall into two families:

dynamical graph systems — Edit Cycles, Degree Rules, and a Conway's Game of Life process — and tree dynamical systems — Boolean Formulae and Peano Addition. Scaling is tested on the arXiv HEP-Th citation network of roughly 27,700 papers, from which a rolling time window carves out 1,554 sub-graphs with node counts $N_G \in [100, 2786]$.

They also flag a hazard in the original setup: training and test series can overlap, giving the model the chance to be evaluated on transitions it saw during training. To address it, the authors propose a stricter risk-estimation protocol that holds out 100 graphs as an independent test set, and re-run the experiments under it.

Results: three of four claims stand, one falls

The original paper makes four core claims: (i) GEN beats VGAE on all three dynamical graph DGPs; (ii) it reaches near-perfect accuracy on the tree DGPs; (iii) forward passes scale sub-quadratically with graph size; and (iv) backward passes scale linearly. The reproduction confirms the first three and refutes the fourth.

The decisive evidence is the slope of a log-log fit of runtime against graph size (a slope below 2 is sub-quadratic; a slope of 1 is linear). Forward-pass slopes come in clearly below 2, so sub-quadratic scaling holds. But the backward-pass slopes, under both edge-filtering schemes, sit clearly above 1 — that is, super-linear. They remain sub-quadratic, but they are not the linear growth the original paper reported.

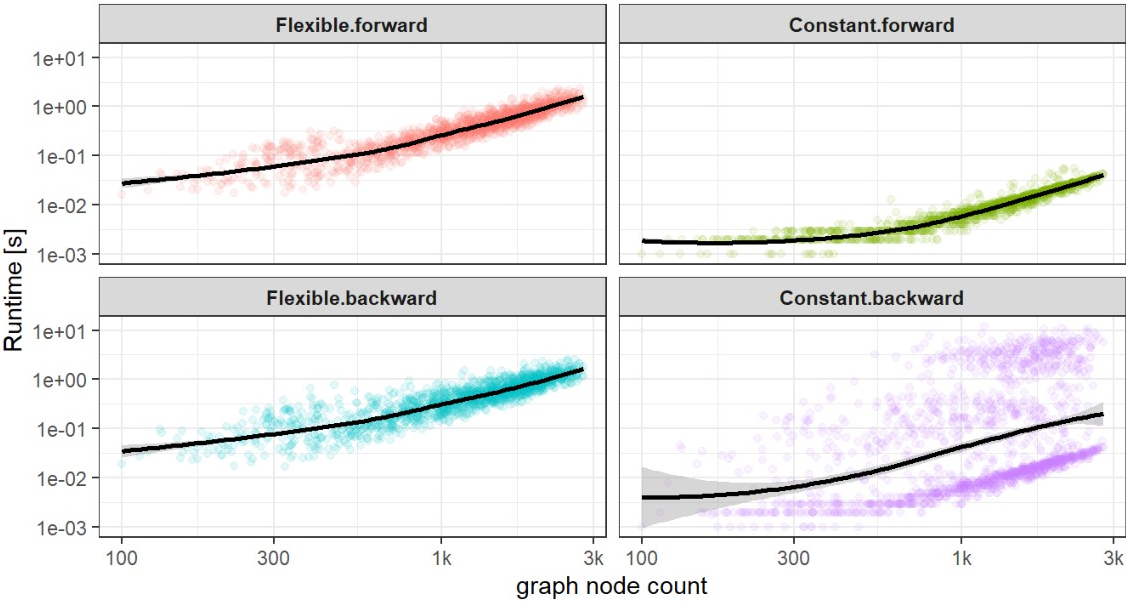


Figure 3. Runtime versus graph scale (x-axis: node count from 100 to 3k; y-axis: runtime; both log-scaled). The four panels show forward and backward passes under Flexible and Constant edge filtering. The black line is the fitted trend: forward passes stay shallow and sub-quadratic, while backward passes rise noticeably faster — especially Constant.backward (bottom right), whose slope is well above 1, directly contradicting the linear-scaling claim.

Table 1. Slopes of the log-log runtime fit (mean ± standard deviation over 5 repetitions). A slope < 2 is sub-quadratic; = 1 is linear. Forward passes are sub-quadratic; backward passes are all above 1 — super-linear — which refutes the original linear-scaling claim.

Filtering	Forward	Backward
Flexible	1.38 ± 0.02	1.30 ± 0.01
Constant	1.31 ± 0.02	1.69 ± 0.10

Fidelity of the reproduction: the numbers largely match

On accuracy, the reproduction is reassuring: the vast majority of metrics fall within one standard deviation of the originally reported values. The largest deviation was an increase in VGAE deletion precision and insertion recall on the Edit Cycles task, which did not overturn GEN's overall win. On the tree tasks, Boolean Formulae accuracy is 0.98 ± 0.02 , and Peano Addition is a perfect 1.0.

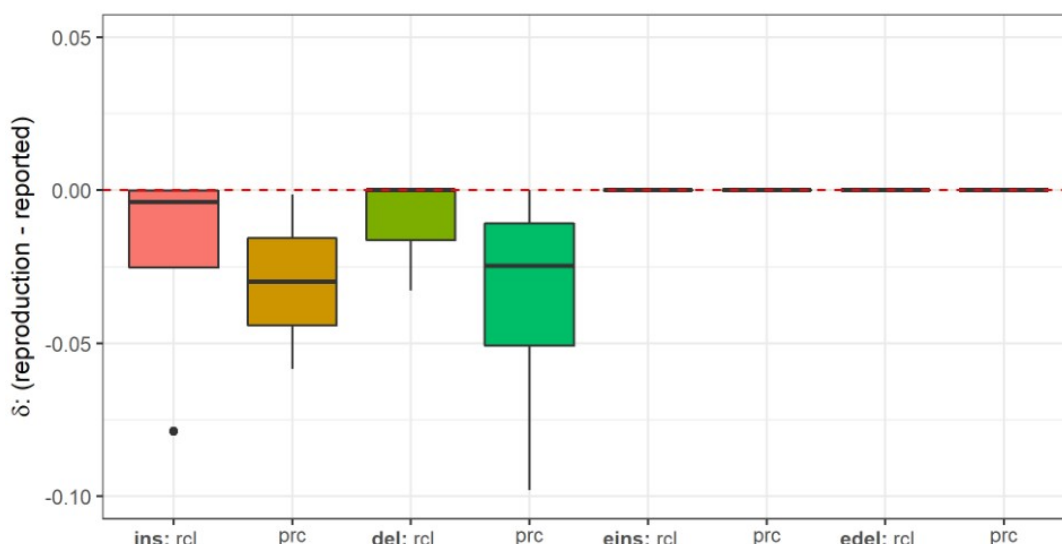


Figure 4. A diagnostic δ -boxplot: the y-axis is (reproduction – reported), with the red dashed line at 0 marking perfect agreement. These results are for the Game of Life DGP with the hinge-loss GEN under the stricter risk-estimation protocol and a larger test set. Most boxes hug 0, with deviations within about 0.05 — the reproduction closely tracks the original.

The authors run two robustness checks. First, they replace the ad-hoc random-graph initialization with the established Erdős–Rényi and Configuration models; every metric moves by less than 0.05. Second, introducing the 100-graph held-out test set shifts scores only slightly, by $\delta \in [-0.1, 0.05]$. Both point to the same conclusion: the original results do not hinge on an experimental shortcut.

The sharper question: are these benchmarks actually hard?

The most interesting result is the last diagnostic. When the authors sample from the tree generators to see what sequences they actually produce, they find a problem: the generators overwhelmingly produce trees that cannot be simplified at all. Across more than 300,000 samples, the probability of drawing a simplifiable tree is only 0.13 ± 0.003 for Boolean Formulae and 0.26 ± 0.002 for Peano Addition.

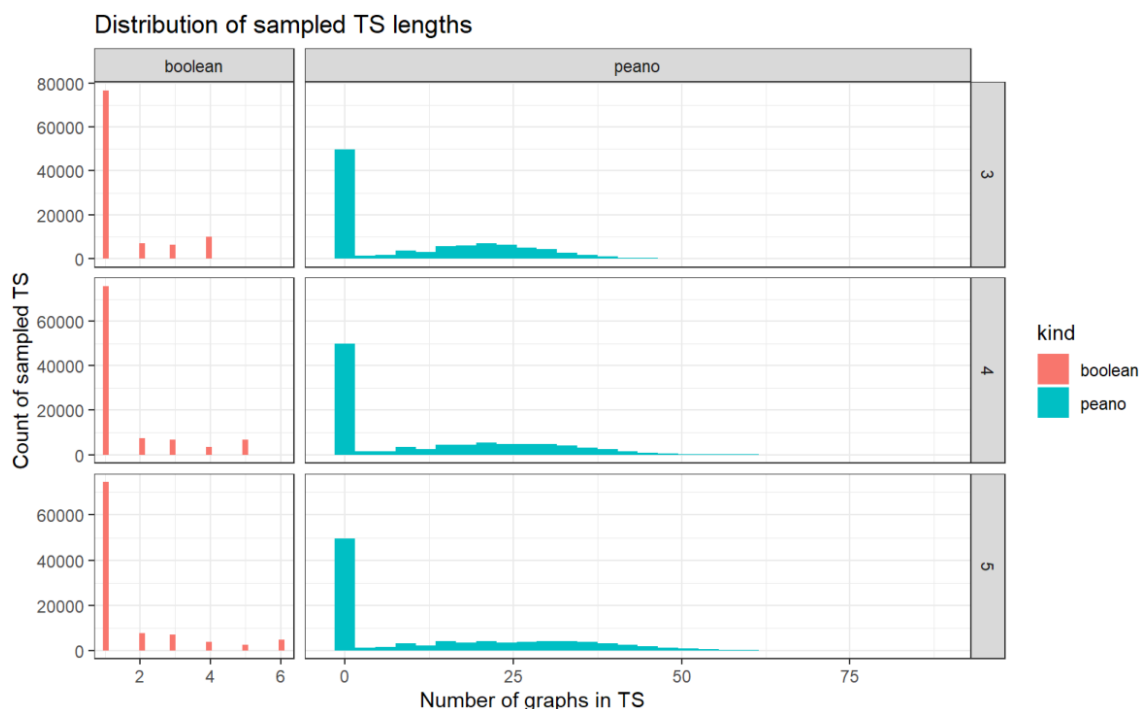


Figure 5. Distribution of sampled time-series lengths from the tree DGPs (boolean and peano; rows correspond to a maximum of 3–5 operations). The mass piles up at very short series: a large share of samples have almost no simplification steps to perform. That undercuts how demanding the near-perfect-accuracy tasks really are.

This puts an asterisk on the tidy 'near-perfect accuracy' numbers on the tree tasks. If most test samples require no simplification in the first place, high accuracy reflects an easy task more than a strongly expressive model. Put differently, the model may be replaying transformations it has already learned rather than generalizing to new ones.

Takeaway: elegant and reproducible, but don't overrate its scaling or its difficulty

Taken together, the reproduction offers a measured verdict. GEN is an elegant, interpretable, and genuinely reproducible architecture, and three of the paper's four claims survive independent scrutiny. But the reproduction corrects one scaling claim — backward passes are super-linear, not linear — and it cautions that several of the synthetic benchmarks let the model win by revisiting transitions it has already learned rather than by generalizing.

It is worth noting that the study's careful documentation of the five data generators may be as valuable as its verdict on the claims: anyone reusing these benchmarks benefits directly from it. The authors close with a call for more rigorous, more practically grounded evaluation of an otherwise promising architecture.